Министерство науки и высшего образования Российской Федерации

Федеральное государственное автономное образовательное
учреждение высшего образования
Национальный исследовательский Нижегородский государственный
университет им. Н.И. Лобачевского

Институт информационных технологий, математики и механики

Отчет по лабораторной работе №1

«Множества на основе бинарных полей»

Выполнил:
студент группы 3825Б1ФИсп1
Кеткова Ю.А.

Проверил:
Ивлев А.Д.

Нижний Новгород
2026

Содержание

Введение 3
1. Постановка задачи 4
2. Руководство пользователя 6
3. Руководство программиста 10
3.1. Описание структуры программы 10
3.2. Описание структур данных 10
3.3. Описание алгоритмов 10
4. Результаты экспериментов и тестирования 12
Заключение 14
Литература 15
Приложение 16

Введение

Битовое поле — это структура данных, которая соответствует одному или нескольким соседним битам, выделенным для определённых целей, так что любой отдельный бит или группа битов в структуре могут быть установлены или проверены. Битовое поле чаще всего используется для представления целочисленных типов с известной фиксированной разрядностью, например однобитовых булевых значений.

Значение отдельных битов в поле определяется программистом. Например, первый бит в битовом поле (расположенный по базовому адресу поля) иногда используется для определения состояния определённого атрибута, связанного с битовым полем.

Битовые поля можно использовать для снижения потребления памяти, когда программе требуется несколько целочисленных переменных, которые всегда будут иметь низкие значения. Например, во многих системах для хранения целочисленного значения требуется два байта (16 бит) памяти; иногда для хранения значений достаточно одного или двух битов. Использование нескольких таких крошечных переменных в одном битовом поле позволяет эффективно упаковывать данные в памяти.

Битовое множество — это абстрактная структура данных, которая хранит набор элементов с помощью битового вектора (массива битов). Идея в том, чтобы каждому элементу множества присвоить уникальный номер (индекс), а затем проверять или устанавливать соответствующий бит: если бит установлен — элемент принадлежит множеству, если сброшен — нет.

Цель работы: разработка структуры данных для хранения множеств с использованием битовых полей и освоение Google Test для автоматического тестирования.

1. Постановка задачи

Выполнение работы предполагает решение следующих задач:

1. Реализация класса битового поля TBitField согласно заданному интерфейсу.
2. Реализация класса множества TSet согласно заданному интерфейсу.
3. Написание и обеспечение работоспособности тестов, покрывающих основной функционал и краевые случаи.

Интерфейсы классов TBitField и TSet находятся в папке include. Объявления классов должны оставаться неизменными (кроме операторов ввода/вывода, для которых допускается inline-реализация), а реализация данных классов может быть вынесена в отдельные cpp-файлы или оставлена в заголовочном файле.

1) Реализация класса битового поля TBitField

Необходимо разработать класс для компактного хранения и обработки последовательности битов произвольной длины. Управление памятью должно осуществляться динамически с помощью массива элементов типа TELEM (unsigned int).

Интерфейс:

1. **Конструкторы и деструктор:**
   - Конструктор, принимающий длину поля в битах, выделяющий динамическую память под массив и инициализирующий все биты нулями. Длина в машинных словах должна вычисляться с учётом остатка от деления.
   - Конструктор копирования, обеспечивающий глубокое копирование.
   - Деструктор, освобождающий динамически выделенную память.
2. **Служебные методы:**
   - GetMemIndex(n) – вычисление индекса элемента массива, содержащего n-й бит.
   - GetMemMask(n) – формирование битовой маски для доступа к n-му биту.
   - GetLength() – возвращение общей длины битового поля в битах.
3. **Операции над битами:**
   - SetBit(n) – установка n-го бита в 1.
   - ClrBit(n) – сброс n-го бита в 0.
   - GetBit(n) – возвращение значения n-го бита.
4. **Перегрузка операторов:**
   - Сравнения: == и !=.
   - Присваивания: = (с обязательной проверкой на самоприсваивание).
   - Побитовые: | (или), & (и), ~ (инверсия). Для операций | и & требуется совпадение длин полей, иначе генерируется std::invalid_argument.
5. **Потоковый ввод-вывод:**
   - operator>> – посимвольный ввод '0' и '1'. При вводе недопустимого символа устанавливается флаг ошибки потока (failbit).
   - operator<< – вывод последовательности '0' и '1', соответствующей состоянию поля.

2) Реализация класса множества TSet

Необходимо разработать класс, реализующий абстракцию математического множества целых неотрицательных чисел, используя класс TBitField в качестве внутреннего представления.

Интерфейс:

1. **Конструкторы:**
   - Конструктор, принимающий максимальную мощность множества.
   - Конструктор копирования.
   - Конструктор преобразования типа из объекта TBitField.
2. **Методы доступа и модификации:**
   - GetMaxPower() – возвращение максимальной мощности множества.
   - InsElem(Elem) – добавление элемента в множество.
   - DelElem(Elem) – удаление элемента из множества.

- IsMember(Elem) – проверка принадлежности элемента множеству.

3. **Перегрузка операторов:**
   - Сравнения: == и !=.
   - Присваивания: = (с проверкой на самоприсваивание).
   - Теоретико-множественные операции, возвращающие новый объект TSet:
     - + с целым числом: возвращает копию множества с добавленным элементом.
     - - с целым числом: возвращает копию множества с удалённым элементом.
     - + с объектом TSet: объединение множеств (требует совпадения MaxPower, иначе std::invalid_argument).
     - * с объектом TSet: пересечение множеств (требует совпадения MaxPower, иначе std::invalid_argument).
     - ~: дополнение множества.
   - Оператор преобразования типа к TBitField.
4. **Потоковый ввод-вывод:**
   - operator>> – считывание характеристического вектора множества в виде строки из '0' и '1' (делегируется TBitField).
   - operator<< – вывод характеристического вектора множества в виде строки из '0' и '1'.

3) Тестирование

Необходимо разработать набор автоматизированных тестов с использованием Google Test, покрывающих основной функционал и проверяющих устойчивость программы к некорректным данным. Тесты должны проверять граничные случаи, работу с памятью и исключения.

2. Руководство пользователя

Программа представляет собой библиотеку классов на языке C++ для работы с множествами целых неотрицательных чисел на основе битовых полей. Основные компоненты:

- TBitField — низкоуровневый класс для хранения и побитовой обработки данных.
- TSet — высокоуровневый класс, реализующий абстракцию математического множества поверх TBitField.

Программа предназначена для автоматизированного тестирования с помощью фреймворка Google Test. Интерактивный ввод с клавиатуры не требуется. Ниже приведены примеры использования классов на основе написанных тестов.

Программное использование класса TBitField

Создание и инициализация битового поля:

```
TEST(TBitField, Constructor)
{
    TBitField bf(10);
    EXPECT_EQ(bf.GetLength(), 10);
}
```

Рис. 1. Тест конструктора и инициализации битового поля

Конструктор TBitField(int len) корректно выделяет память под заданное число бит. Все биты после создания равны 0.

Установка и получение значений битов:

```
TEST(TBitField, SetBit)
{
    TBitField bf(5);
    bf.SetBit(2);
    EXPECT_EQ(bf.GetBit(2), 1);
}
```

Рис. 2. Тест установки и получения значений битов

Метод SetBit(n) устанавливает n-й бит в 1. Метод GetBit(n) корректно возвращает значение бита.

Сброс битов:

```
TEST(TBitField, ClearBit)
{
    TBitField bf(5);
    bf.SetBit(2);
    bf.ClrBit(2);
    EXPECT_EQ(bf.GetBit(2), 0);
}
```

Рис. 3. Тест сброса битов

Метод ClrBit(n) сбрасывает только указанный бит в 0.

Конструктор копирования и оператор присваивания:

```
TEST(TBitField, CopyConstructor)
{
    TBitField bf(5);
    bf.SetBit(1);
    TBitField copy(bf);
    EXPECT_EQ(copy, bf);
}
```

Рис. 4. Тест конструктора копирования и оператора присваивания

Конструктор копирования создаёт независимую копию. Оператор = корректно копирует содержимое. Проверка самоприсваивания (bf = bf) не приводит к потере данных.

Побитовые операции:

```
TEST(TBitField, BitwiseOr)
{
    TBitField bf1(4);
    TBitField bf2(4);
    bf1.SetBit(0);
    bf2.SetBit(2);

    TBitField result = bf1 | bf2;

    EXPECT_EQ(result.GetBit(0), 1);
    EXPECT_EQ(result.GetBit(2), 1);
}
```

Рис. 5. Тест побитовых операций

Оператор | выполняет побитовое ИЛИ. Оператор & — побитовое И. Оператор ~ инвертирует все биты в пределах заданной длины (с маскировкой лишних битов в последнем слове).

Ввод/вывод и обработка ошибок:

```
TEST(TBitField, InputAndOutput)
{
    TBitField bf(5);

    std::istringstream input("10101");
    input >> bf;

    EXPECT_TRUE(input.good() || input.eof());

    EXPECT_EQ(bf.GetBit(4), 1);
    EXPECT_EQ(bf.GetBit(3), 0);
    EXPECT_EQ(bf.GetBit(2), 1);
    EXPECT_EQ(bf.GetBit(1), 0);
    EXPECT_EQ(bf.GetBit(0), 1);

    std::ostringstream output;
    output << bf;

    EXPECT_EQ(output.str(), "10101");
}

TEST(TBitField, InputInvalidCharacter)
{
    TBitField bf(3);

    std::istringstream input("10x");
    input >> bf;

    EXPECT_TRUE(input.fail());
}
```

Рис. 6. Тест потокового ввода-вывода и обработки ошибок

При вводе недопустимого символа (не '0' и не '1') поток переходит в состояние ошибки, но исключение не выбрасывается.

Программное использование класса TSet

Создание множества и базовые операции:

```
TEST(TSet, InsertElement)
{
    TSet s(10);
    s.InsElem(3);
    EXPECT_EQ(s.IsMember(3), 1);
}
```

Рис. 7. Тест создания множества и базовых операций с элементами

Конструктор TSet(int mp) задаёт максимальную мощность. Методы InsElem и DelElem изменяют состояние множества. IsMember проверяет принадлежность.

Теоретико-множественные операции:

```
TEST(TSet, Union)
{
    TSet s1(10);
    TSet s2(10);
    s1.InsElem(1);
    s2.InsElem(2);

    TSet result = s1 + s2;

    EXPECT_EQ(result.IsMember(1), 1);
    EXPECT_EQ(result.IsMember(2), 1);
}
```

Рис. 8. Тест теоретико-множественных операций

Операторы + и * для двух множеств работают только при совпадении MaxPower. В противном случае выбрасывается std::invalid_argument (проверяется в тестах UnionDifferentPowersThrows и IntersectionDifferentPowersThrows).

Операции с отдельными элементами:

```
TEST(TSet, UnionWithElement)
{
    TSet s(5);
    s.InsElem(1);

    TSet result = s + 3;

    EXPECT_EQ(result.IsMember(1), 1);
    EXPECT_EQ(result.IsMember(3), 1);
    EXPECT_EQ(s.IsMember(3), 0);
}
```

Рис. 9. Тест операций с отдельными элементами множества

Операторы + int и - int возвращают новый объект, не модифицируя исходный.

Дополнение множества:

```
TEST(TSet, Complement)
{
    TSet s(4);
    s.InsElem(1);
    s.InsElem(3);

    TSet result = ~s;

    EXPECT_EQ(result.IsMember(0), 1);
    EXPECT_EQ(result.IsMember(1), 0);
    EXPECT_EQ(result.IsMember(2), 1);
    EXPECT_EQ(result.IsMember(3), 0);
}
```

Рис. 10. Тест операции дополнения множества

Оператор ~ инвертирует биты строго в пределах MaxPower благодаря маскировке в TBitField::operator~.

Ввод/вывод:

```
TEST(TSet, InputAndOutput)
{
    TSet s(5);

    std::istringstream input("10101");
    input >> s;

    EXPECT_TRUE(input.good() || input.eof());

    EXPECT_EQ(s.IsMember(4), 1);
    EXPECT_EQ(s.IsMember(3), 0);
    EXPECT_EQ(s.IsMember(2), 1);
    EXPECT_EQ(s.IsMember(1), 0);
    EXPECT_EQ(s.IsMember(0), 1);

    std::ostringstream output;
    output << s;

    EXPECT_EQ(output.str(), "10101");
}
```

Рис. 11. Тест потокового ввода-вывода для класса множества

Ввод и вывод TSet полностью делегируются TBitField, представляя множество в виде строки из '0' и '1'.

Запуск автоматических тестов

1. Скомпилируйте проект вместе с файлами тестов.
2. Запустите исполняемый файл.
3. Фреймворк автоматически выполнит все сценарии:
    - TBitField: проверка корректности работы (инициализация, операции, копирование, исключения).
    - TSet: проверка корректности работы (добавление, удаление, теоретико-множественные операции, исключения).

Успешное выполнение подтверждается сообщением фреймворка:

```
[----------] Global test environment tear-down
[==========] 50 tests from 2 test cases ran. (26 ms total)
[  PASSED  ] 50 tests.
```

Рис. 12. Сообщение фреймворка об успешном выполнении

3. Руководство программиста

3.1. Описание структуры программы

Проект состоит из следующих модулей:

1. **Модуль битового поля**: реализация низкоуровневой структуры данных. Состав: класс TBitField, глобальные перегруженные операторы >> и <<.
2. **Модуль множества**: реализация высокоуровневой абстракции. Состав: класс TSet, глобальные перегруженные операторы >> и <<.
3. **Модуль тестирования**: автоматизированная проверка. Состав: наборы тестов TBitField и TSet на базе Google Test.
4. **Заголовочные файлы интерфейсов** (includes/Lab1/): хранение объявлений классов.

3.2. Описание структур данных

Класс TBitField:

- int BitLen — общая длина поля в битах.
- int MemLen — количество машинных слов.
- TELEM* pMem — указатель на динамический массив слов (TELEM = unsigned int).

Класс TSet:

- int MaxPower — максимальная мощность множества.
- TBitField BitField — внутреннее битовое поле (характеристический вектор).

3.3. Описание алгоритмов

1. Алгоритм адресации бита:

- Индекс машинного слова: index = n / (sizeof(TELEM) * 8).
- Позиция бита в слове: r = n % (sizeof(TELEM) * 8).
- Маска: mask = (TELEM)1 << r.

2. Алгоритм выделения памяти: Количество слов вычисляется с округлением вверх: MemLen = (len + sizeof(TELEM) * 8 - 1) / (sizeof(TELEM) * 8). После выделения памяти (new TELEM[MemLen]()) массив инициализируется нулями.

3. Алгоритм инверсии битового поля (TBitField::operator~**):**

1. Все элементы массива инвертируются побитово (~pMem[i]).
2. Если длина поля не кратна размеру слова, вычисляется количество используемых битов в последнем слове: usedBits = BitLen % (sizeof(TELEM) * 8).
3. Формируется маска для обнуления «мусорных» старших битов: mask = (1 << usedBits) - 1.
4. Последний элемент массива логически умножается на маску: result.pMem[MemLen - 1] &= mask.

4. Алгоритмы операций над множествами (TSet**):**

- **Объединение (+) и пересечение (*):** проверяют равенство MaxPower. Если мощности различаются, генерируется std::invalid_argument. Иначе создаётся новое множество, и в цикле от 0 до MaxPower - 1 проверяются условия (логическое ИЛИ или И для битов).

- **Дополнение (~):** вызывает TSet(~BitField). Благодаря алгоритму инверсии в TBitField, в результирующем множестве не появляются элементы за пределами MaxPower.

5. Алгоритм потокового ввода/вывода:

- TBitField::operator>>: считывает символы. Если символ не '0' и не '1', устанавливает failbit и возвращает поток. Если '1' — устанавливает бит, если '0' — сбрасывает. Цикл идёт от старшего бита (BitLen - 1) к младшему (0).

- TSet::operator>> и <<: делегируют вызовы внутреннему BitField.

4. Результаты экспериментов и тестирования

Для проверки корректности работы разработанных классов было проведено тестирование с использованием фреймворка Google Test. Написано 50 тестов, разделённых на две логические группы.

1. Эксперименты с классом TBitField

- **Инициализация:** проверяется, что память выделяется корректно (включая нулевую длину и длину, пересекающую границу машинного слова), а все биты инициализируются нулями.
- **Работа с битами:** проверяются установка, сброс и чтение битов на границах (первый и последний биты). Тесты NegativeIndexThrows, IndexEqualToLengthThrows и GetBitOutOfRangeThrows подтверждают, что при выходе за границы генерируется std::out_of_range.
- **Управление памятью:** тесты копирования и присваивания (CopyIsIndependent, SelfAssignment, AssignmentDifferentLengths) подтверждают глубокое копирование и безопасность при самоприсваивании.
- **Побитовые операции:** проверяются |, &, ~. Тесты OrDifferentLengthsThrows и AndDifferentLengthsThrows подтверждают выброс std::invalid_argument при несовпадении длин.
- **Ввод/вывод:** InputAndOutput проверяет корректность чтения и записи строки "10101". InputInvalidCharacter подтверждает установку флага failbit при встрече недопустимого символа 'x'.

2. Эксперименты с классом TSet

- **Базовые операции:** добавление, удаление и проверка элементов. Тесты InsertOutOfRangeThrows и DeleteOutOfRangeThrows подтверждают выброс std::out_of_range при выходе за границы MaxPower.
- **Теоретико-множественные операции:** проверяются объединение и пересечение. Тесты UnionDifferentPowersThrows и IntersectionDifferentPowersThrows подтверждают выброс std::invalid_argument при разных мощностях. Тест Complement проверяет корректность инверсии в пределах MaxPower.
- **Операции с элементами:** тесты UnionWithElement и DifferenceWithElement подтверждают, что операторы + int и - int возвращают новые объекты, не изменяя исходные.
- **Ввод/вывод:** InputAndOutput проверяет чтение и запись строки "10101".

Анализ результатов:

По итогам запуска фреймворка Google Test все 50 написанных тестов завершились успешно. Детальный анализ прохождения тестов позволяет сделать следующие выводы о работе реализованных классов:

1. Корректность управления динамической памятью (TBitField**)**

Тесты, связанные с копированием и присваиванием, подтвердили, что класс корректно управляет ресурсами.

- Тест CopyIsIndependent показал, что конструктор копирования выполняет глубокое копирование: изменение битов в скопированном объекте не приводит к изменению оригинального поля.
- Тест SelfAssignment подтвердил, что оператор присваивания безопасно обрабатывает самоприсваивание (благодаря использованию идиомы copy-and-swap в реализации), не вызывая утечек памяти или потери данных.
- Тест AssignmentDifferentLengths доказал, что оператор присваивания корректно перераспределяет память, если длины исходного и целевого полей различаются.

2. Работа с битами и граничные случаи (TBitField**)**

Тесты базовых операций выявили, что алгоритмы адресации битов работают верно на всех участках поля.

- Тест FirstAndLastBit подтвердил корректную установку крайних битов (индексы 0 и BitLen - 1).
- Тест LengthAcrossMemoryBoundary показал, что переход от одного машинного слова к другому (когда индекс бита превышает sizeof(TELEM) * 8) обрабатывается правильно, без выхода за границы массива.
- Тест Complement подтвердил эффективность алгоритма инверсии. Благодаря использованию битовой маски для последнего машинного слова, при инверсии поля (например, длины 4) в результате не появляются «мусорные» биты на позициях от 4 до 31.

3. Обработка исключительных ситуаций при работе с индексами

Тесты показали, что классы защищены от выхода за допустимые границы.

- Тесты NegativeIndexThrows, IndexEqualToLengthThrows и GetBitOutOfRangeThrows подтвердили, что при попытке доступа к несуществующему биту в TBitField корректно генерируется исключение std::out_of_range.
- Аналогично, тесты InsertOutOfRangeThrows и DeleteOutOfRangeThrows в классе TSet подтверждают, что попытка добавить или удалить элемент с индексом ≥MaxPower приводит к выбросу std::out_of_range.

4. Логика теоретико-множественных операций (TSet**)**

- Тесты Union, Intersection и Complement подтвердили, что математические операции над множествами работают согласно ожиданиям и корректно делегируются побитовым операциям внутреннего TBitField.
- Важным результатом стала проверка ограничений на операции между разными множествами. Тесты UnionDifferentPowersThrows и IntersectionDifferentPowersThrows показали, что при попытке объединить или пересечь множества с разной максимальной мощностью программа не выполняет некорректных действий, а генерирует исключение std::invalid_argument.
- Тесты UnionWithElement и DifferenceWithElement подтвердили, что операторы + int и - int работают по принципу «иммутабельности»: они возвращают новую копию множества, оставляя исходный объект без изменений.

5. Устойчивость потокового ввода

Тест InputInvalidCharacter показал, что при попытке ввести символ, отличный от '0' или '1', программа не завершается аварийно. Вместо этого оператор >> выставляет флаг std::ios::failbit, переводя поток в состояние ошибки. Это позволяет вызывающему коду обнаружить проблему и корректно обработать её.

Общий вывод:

Результаты тестирования подтверждают, что разработанные классы TBitField и TSet работают стабильно, не вызывают неопределённого поведения при выходе за границы или некорректном вводе, а их функционал полностью соответствует заявленному интерфейсу. Использование Google Test позволило автоматизировать проверку и убедиться в том, что внесение изменений в код не нарушает существующую логику работы программы.

Заключение

В ходе выполнения лабораторной работы была достигнута поставленная цель: разработана структура данных для хранения множеств на основе битовых полей и освоен фреймворк Google Test для автоматического тестирования.

В результате работы были выполнены следующие задачи:

1. Реализован класс `TBitField`, предназначенный для хранения последовательностей битов произвольной длины. Класс использует динамическое выделение памяти под массив машинных слов. Реализованы конструкторы (включая копирования), деструктор и оператор присваивания для корректной работы с памятью. Также реализованы базовые операции установки, сброса и чтения битов, побитовые операции и потоковый ввод/вывод.

2. Создан класс `TSet`, реализующий множество целых неотрицательных чисел через композицию с `TBitField`. Класс поддерживает добавление и удаление элементов, проверку принадлежности, а также основные теоретико-множественные операции: объединение, пересечение, разность и дополнение. Для операций над двумя множествами реализована проверка на совпадение их максимальных мощностей.

3. Разработан набор тестов с использованием Google Test. Написано 50 тестов, которые проверяют базовый функционал классов, работу с памятью (копирование и присваивание), граничные случаи (например, нулевые индексы или индексы на стыке машинных слов), а также обработку исключений (`std::out_of_range` и `std::invalid_argument`). Все тесты успешно проходят.

В процессе выполнения работы были закреплены навыки работы с динамической памятью в C++, перегрузкой операторов и написания unit-тестов. Разработанные классы соответствуют заданному интерфейсу, а их работа проверена автоматическими тестами. Поставленные в начале работы задачи выполнены.

Литература

Список источников, используемых при выполнении работы (если есть).

1. Страуструп Бьерн Язык программирования C++. Специальное издание. Пер. с англ. – М.: Издательство Бином, 2017 г. – 1136 с.
2. Сайт cppreference: https://en.cppreference.com/
3. Информация о битовых полях: https://ru.wikipedia.org/wiki/Битовое_поле

Приложение

Фрагмент реализации TBitField::operator~ **(маскировка неиспользуемых битов):**

tbitfield.cpp

```
TBitField TBitField::operator~(void)
{
    TBitField result(BitLen);
    for (int i = 0; i < MemLen; ++i)
        result.pMem[i] = ~pMem[i];

    const int bitsPerElement = sizeof(TELEM) * 8;
    const int usedBits = BitLen % bitsPerElement;

    if (MemLen > 0 && usedBits != 0)
    {
        TELEM mask = (static_cast<TELEM>(1) << usedBits) - 1;
        result.pMem[MemLen - 1] &= mask;
    }
    return result;
}
```

Фрагмент реализации TBitField::operator>>**:**

tbitfield.cpp

```
std::istream& operator>>(std::istream& istr, TBitField& bf)
{
    for (int i = bf.BitLen - 1; i >= 0; --i)
    {
        char ch;
        if (!(istr >> ch))
            return istr;
        if (ch != '0' && ch != '1')
        {
            istr.setstate(std::ios::failbit);
            return istr;
        }
        if (ch == '1') bf.SetBit(i);
        else bf.ClrBit(i);
    }
    return istr;
}
```